製造業見積エンジン — 3層アーキテクチャ設計書

> **ステータス**: 設計完了・未実装
> **目的**: 製造業全24中分類を1つのエンジンでカバーする
> **設計根拠**: `e_02_製造業BPO設計.md` Section 15.6b（3層ナレッジモデル）

設計思想

```
Layer 0: LLMデフォルト — 何も設定がなくても概算が出る（全24業種Day 1対応）
Layer 1: YAML設定      — 推定を事実に置き換えて精度を上げる（業種データ固定）
Layer 2: Pythonプラグイン — 計算の公式自体が違う業種に対応（例外のみ）

上のレイヤーがなければ下にフォールバック。重ねて使う。
```

解決順序（各データ項目ごと）

```
チャージレート: 顧客DB → YAML → LLM推定
材料単価:       顧客DB → YAML → LLM推定
工程リスト:     Plugin → YAMLルール → LLM推定
サイクルタイム: 顧客の過去実績 → YAML → LLM推定
```

Confidenceスコア

ソース	Confidence
LLMのみ（Layer 0）	0.3-0.5
YAML設定あり（Layer 1）	0.6-0.8
YAML + 顧客DB	0.8-0.9
Plugin（Layer 2）	0.7-0.9
過去実績あり	0.9-1.0

ディレクトリ構成

```
shachotwo-app/
├── workers/bpo/manufacturing/
│   ├── engine.py                    # 3層オーケストレーター（本設計の中核）
│   ├── models.py                    # 拡張（HearingInput, QuoteResult等を追加）
│   ├── quoting.py                   # 既存 → engine.pyへ段階移行
│   ├── plugins/                     # Layer 2 Pythonプラグイン
│   │   ├── __init__.py              #   プラグインレジストリ
│   │   ├── base.py                  #   ABC定義
│   │   ├── plastics.py              #   樹脂成型（金型償却）
│   │   ├── food_chemical.py         #   食品・化学（配合展開）
│   │   └── electronics.py           #   電子部品（BOM組立）
│   └── pipelines/
│       └── quoting_pipeline.py      # engine.pyを呼ぶように変更
│
├── brain/genome/data/
│   └── manufacturing_quoting/       # Layer 1 YAML設定ファイル
│       ├── _schema.yaml             #   スキーマ定義
│       ├── metalwork.yaml           #   金属加工（現ハードコードから移行）
│       ├── plastics.yaml            #   樹脂成型
│       ├── food_chemical.yaml       #   食品・化学
│       ├── electronics.yaml         #   電子部品
│       └── ...                      #   必要に応じて追加
│
└── db/migrations/
    └── 015_mfg_quoting_engine.sql   #   スキーマ拡張
```

新業種の追加手順

何もしない（Layer 0: 即日）

```
ユーザーが sub_industry = "textile" で見積リクエスト
  → Plugin? → ない
  → YAML?   → ない
  → LLM が「繊維製造」のコンテキストで工程推定（精度±30-50%）
```

作業: ゼロ。24業種すべてDay 1から概算が出せる。

YAML追加（Layer 1: 数時間）

```
1. brain/genome/data/manufacturing_quoting/textile.yaml を1ファイル作成
2. 材料・設備・工程ルーティングを定義
3. 精度: ±15-25%
```

Plugin追加（Layer 2: 数日）

```
1. workers/bpo/manufacturing/plugins/textile.py を作成
2. ManufacturingPlugin を継承、estimate_processes() を実装
3. 精度: ±10-20%
```

各レイヤーが必要な業種マップ

中分類	業種	Layer 0	Layer 1	Layer 2	理由
24-27	金属加工・機械	○	◎ metalwork.yaml	不要	形状→工程のルーティングで十分
18-19	プラスチック・ゴム	○	◎ plastics.yaml	**必要**	金型償却の計算ロジックが特殊
09-10	食品・飲料	○	○ food_chemical.yaml	**必要**	配合展開（レシピ→原料量）が特殊
16	化学	○	○ food_chemical.yaml	**必要**	バッチ計算が特殊
28-29	電子部品・電気機械	○	○ electronics.yaml	**必要**	BOM展開が特殊
11	繊維	○	△（あれば精度UP）	不要	
12-13	木材・家具	○	△	不要	
14	パルプ・紙	○	△	不要	
15	印刷	○	△	不要	
17	石油・石炭	○	×	不要	ニッチ。LLMで十分
20	なめし革	○	×	不要	ニッチ
21	窯業・土石	○	△	不要	
22-23	鉄鋼・非鉄金属	○	◎ metalwork.yaml	不要	金属加工と同じYAMLで対応
30-32	情報通信・輸送・その他	○	×	不要	大手中心。LLMで概算

Layer 2（Pythonプラグイン）が必要なのは4業種グループだけ。残り20業種はYAML + LLMで対応。

エンジン設計（engine.py）

クラス構造

```
# DESIGN — 実装コードではない

class ManufacturingQuotingEngine:

    async def run(
        self,
        company_id: str,
        hearing: HearingInput,
    ) -> QuoteResult:
        sub_industry = self._resolve_sub_industry(hearing)
        plugin = get_plugin(sub_industry)        # Layer 2
        yaml_config = load_yaml_config(sub_industry)  # Layer 1
        customer = await self._load_customer_data(company_id)  # 顧客DB

        # 工程推定（解決順序: Plugin → YAML → LLM）
        if plugin and plugin.can_estimate_processes(hearing):
            processes = await plugin.estimate_processes(hearing, yaml_config, customer)
        elif yaml_config and yaml_config.get("process_routing_rules"):
            processes = self._estimate_from_yaml(hearing, yaml_config, customer)
        else:
            processes = await self._estimate_from_llm(hearing, sub_industry)

        # コスト計算（共通。チャージレート等は顧客DB→YAML→LLMの順で解決）
        costs = self._calculate_costs(processes, hearing, yaml_config, customer)

        # プラグイン固有の追加コスト（金型償却等）
        additional = []
        if plugin:
            additional = plugin.calculate_additional_costs(hearing, processes)

        quote_id = await self._save_quote(company_id, hearing, processes, costs,
additional)
        return QuoteResult(quote_id=quote_id, processes=processes, costs=costs,
additional_costs=additional)
```

コスト計算（全レイヤー共通の公式）

```
材料費 = 材料単価 × 使用量 × ロス率 × 数量
加工費 = Σ（チャージレート ×（段取り時間 + サイクルタイム × 数量）/ 60）
外注費 = Σ（基本費 + 個あたり費 × 数量）
追加費 = Σ（プラグイン固有。金型償却、配合ロス等）
管理費 =（材料費 + 加工費 + 外注費）× 管理費率
利益   =（材料費 + 加工費 + 外注費 + 管理費）× 利益率
合計   = 材料費 + 加工費 + 外注費 + 追加費 + 管理費 + 利益
```

この公式は全24業種で共通。 各項目の「値」がレイヤーによって異なるだけ。

YAMLスキーマ

最小構成（新業種追加時のテンプレート）

```yaml
id: new_industry
name: 業種名（日本語）
description: 説明
jsic_codes: ["E-XX"]
plugin: null  # Pythonプラグインが不要ならnull

materials:
  material_code:
    name: 表示名
    kg_price: 円/kg

equipment:
  equipment_type:
    name: 表示名
    charge_rate_yen_hour: 円/時間
    setup_time_min: 段取り時間（分）
    cycle_time_base_min: 基準サイクルタイム（分/個）

process_routing_rules: {}  # 空ならLLMにフォールバック

cost_params:
  default_overhead_rate: 0.15
  default_profit_rate: 0.15
```

金属加工YAML（現ハードコードからの移行）

```yaml
id: metalwork
name: 金属加工
description: 切削・板金・研磨・溶接を中心とする金属部品加工
jsic_codes: ["E-22", "E-23", "E-24", "E-25", "E-26"]
plugin: null

materials:
  SS400:  { name: 一般構造用鋼, density_kg_m3: 7850, machinability: 1.0, kg_price: 120 }
  S45C:   { name: 機械構造用炭素鋼, density_kg_m3: 7850, machinability: 0.9, kg_price: 150
}
  SUS304: { name: ステンレス, density_kg_m3: 7930, machinability: 0.6, kg_price: 400 }
  SUS316: { name: ステンレス316, density_kg_m3: 7980, machinability: 0.55, kg_price: 500 }
  A5052:  { name: アルミ合金, density_kg_m3: 2680, machinability: 1.3, kg_price: 500 }
  A6061:  { name: アルミ合金, density_kg_m3: 2700, machinability: 1.2, kg_price: 550 }
  A7075:  { name: 超々ジュラルミン, density_kg_m3: 2800, machinability: 1.0, kg_price: 800
}
  C3604:  { name: 快削黄銅, density_kg_m3: 8470, machinability: 1.5, kg_price: 1200 }
  POM:    { name: ポリアセタール, density_kg_m3: 1410, machinability: 1.5, kg_price: 800 }
  PEEK:   { name: PEEK樹脂, density_kg_m3: 1300, machinability: 0.8, kg_price: 15000 }

equipment:
  cutting:           { name: 材料切断, charge_rate_yen_hour: 3000, setup_time_min: 15,
cycle_time_base_min: 3 }
  cnc_lathe:         { name: CNC旋盤, charge_rate_yen_hour: 6000, setup_time_min: 30,
cycle_time_base_min: 8 }
  machining_center:  { name: マシニングセンタ, charge_rate_yen_hour: 8000, setup_time_min:
45, cycle_time_base_min: 15 }
  5axis_mc:          { name: 5軸MC, charge_rate_yen_hour: 12000, setup_time_min: 60,
cycle_time_base_min: 20 }
  grinder:           { name: 研磨盤, charge_rate_yen_hour: 6000, setup_time_min: 30,
cycle_time_base_min: 10 }
  laser:             { name: レーザー加工機, charge_rate_yen_hour: 8000, setup_time_min:
20, cycle_time_base_min: 5 }
  wire_cut:          { name: ワイヤーカット, charge_rate_yen_hour: 10000, setup_time_min:
30, cycle_time_base_min: 15 }
  welding:           { name: 溶接, charge_rate_yen_hour: 5000, setup_time_min: 20,
cycle_time_base_min: 10 }
  deburring:         { name: バリ取り, charge_rate_yen_hour: 2500, setup_time_min: 5,
cycle_time_base_min: 3 }
  inspection:        { name: 検査, charge_rate_yen_hour: 3000, setup_time_min: 10,
cycle_time_base_min: 2 }

process_routing_rules:
  round:
    base_processes:
      - { equipment_type: cutting, setup_time_min: 15, cycle_time_factor: 1.0 }
      - { equipment_type: cnc_lathe, setup_time_min: 30, cycle_time_factor: 1.0 }
    conditional_processes:
      - { condition: has_holes, equipment_type: machining_center, setup_time_min: 30,
```

```yaml
cycle_time_factor: 0.33 }
      - { condition: tight_tolerance, equipment_type: grinder, setup_time_min: 30,
cycle_time_factor: 1.0 }
  block:
    base_processes:
      - { equipment_type: cutting, setup_time_min: 15, cycle_time_factor: 1.0 }
      - { equipment_type: machining_center, setup_time_min: 45, cycle_time_factor: 1.0 }
  plate:
    base_processes:
      - { equipment_type: laser, setup_time_min: 20, cycle_time_factor: 1.0 }
  complex:
    base_processes:
      - { equipment_type: cutting, setup_time_min: 15, cycle_time_factor: 1.0 }
      - { equipment_type: machining_center, setup_time_min: 60, cycle_time_factor: 1.33
}
      - { equipment_type: wire_cut, setup_time_min: 30, cycle_time_factor: 1.0 }

outsource_costs:
  heat_treatment: { base_cost: 5000, per_piece_cost: 200 }
  surface_treatment:
    メッキ:     { base_cost: 3000, per_piece_cost: 500 }
    アルマイト:  { base_cost: 3000, per_piece_cost: 300 }
    黒染め:     { base_cost: 2000, per_piece_cost: 100 }

cost_params:
  default_overhead_rate: 0.15
  default_profit_rate: 0.15
  rush_multiplier: 1.2
  prototype_multiplier: 1.5
  lot_discount_thresholds:
    - { min_quantity: 100, discount_rate: 0.05 }
    - { min_quantity: 500, discount_rate: 0.10 }
    - { min_quantity: 1000, discount_rate: 0.15 }
```

Pythonプラグインインターフェース

ABC（plugins/base.py）

```
# DESIGN

class ManufacturingPlugin(ABC):
    """計算の公式自体が違う業種のみ実装する"""

    @property
    @abstractmethod
    def sub_industry_id(self) -> str: ...

    def can_estimate_processes(self, hearing: HearingInput) -> bool:
        return True

    @abstractmethod
    async def estimate_processes(
        self, hearing: HearingInput, yaml_config: dict | None, customer:
CustomerOverrides,
    ) -> list[ProcessEstimate]: ...

    def calculate_additional_costs(
        self, hearing: HearingInput, processes: list[ProcessEstimate],
    ) -> list[AdditionalCostItem]:
        return []  # デフォルト: 追加費用なし
```

Pluginが必要な理由（業種ごと）

業種	特殊な計算	なぜYAMLだけでは無理か
樹脂成型	金型費 ÷ 生産予定数量 = 個あたり金型償却費	「償却」という概念がYAMLの工程ルーティングでは表現できない
食品・化学	レシピ配合比 × バッチサイズ → 各原料の必要量	「配合展開」は材料マスタの参照ではなく逆算ロジック
電子部品	BOMツリー展開 → 部品費 + 実装費の積上げ	「親子関係のある部品表」はフラットなYAMLでは表現困難

LLMデフォルトレイヤー（Layer 0）

プロンプト構造

```
あなたは{業種名}の見積エキスパートです。
以下のヒアリング情報をもとに、製造工程・材料費・加工費を推定してください。

[ヒアリング回答6項目]

出力: JSON形式（processes, material_estimate, outsource_items, confidence）

ガイドライン:
- confidenceは正直に。根拠が薄ければ0.3
- 業界の一般的な相場を使い、notesに根拠を記載
- 工程は10以内
```

YAMLがある場合のLLMの役割縮小

```
YAMLなし: LLMが全て推定（工程リスト、材料単価、チャージレート）
YAMLあり: LLMは「工程リストの推定」だけ担当。単価はYAMLから取得
顧客DBあり: LLMは「形状分類」だけ担当。工程も単価もDBから取得
```

顧客データによるオーバーライド

データ構造

```
# DESIGN

class CustomerOverrides:
    charge_rates: dict[str, int]          # 設備名 → チャージレート
    material_prices: dict[str, int]       # 材料コード → kg単価
    historical_averages: dict[str, dict]  # 設備名 → {avg_setup, avg_cycle, count}
    overhead_rate: float | None
    profit_rate: float | None
```

学習ループ

```
1. AIが見積生成（precision 0.4-0.7）
2. 顧客が修正（「CNC旋盤の段取りは25分」）
3. 修正データをDBに保存（user_modified = true）
4. 次回同じ設備×材質の組み合わせ → 過去実績の平均値を使用
5. 3件以上の実績 → confidence 0.9
```

匿名集約（Phase 2+）

```
全社のmfg_charge_ratesを匿名集約
  → 同じ equipment_type × sub_industry の中央値を計算
  → metalwork.yaml の charge_rate_yen_hour を更新候補として提示
  → 管理者レビュー後にYAMLにマージ
```

DBスキーマ変更（015_mfg_quoting_engine.sql）

```sql
-- mfg_quotes に業種カラム追加
ALTER TABLE mfg_quotes
  ADD COLUMN IF NOT EXISTS sub_industry TEXT DEFAULT 'metalwork',
  ADD COLUMN IF NOT EXISTS layers_used JSONB DEFAULT '[]',
  ADD COLUMN IF NOT EXISTS overall_confidence DECIMAL(3,2),
  ADD COLUMN IF NOT EXISTS additional_costs JSONB DEFAULT '[]';

-- mfg_quote_items にレイヤーソース追加
ALTER TABLE mfg_quote_items
  ADD COLUMN IF NOT EXISTS layer_source TEXT DEFAULT 'yaml';

-- 学習ループ用の集約ビュー
CREATE OR REPLACE VIEW mfg_historical_averages AS
SELECT
  company_id, equipment_type,
  COUNT(*) as sample_count,
  AVG(setup_time_min) FILTER (WHERE user_modified = true) as avg_setup_min,
  AVG(cycle_time_min) FILTER (WHERE user_modified = true) as avg_cycle_min
FROM mfg_quote_items
WHERE user_modified = true AND created_at > now() - interval '6 months'
GROUP BY company_id, equipment_type
HAVING COUNT(*) >= 3;
```

移行計画

Phase	内容	工数	既存への影響
1	engine.py新規作成 + metalwork.yaml移行 + models.py拡張	1週間	**なし**（既存quoting.pyは温存）
2	quoting_pipeline.pyをengine.py呼び出しに切り替え	2-3日	quoting.pyを非推奨化
3	追加プラグイン（plastics/food/electronics）+ YAML	各2-3日	なし（追加のみ）
4	匿名集約・YAMLフィードバック	Phase 2+	なし